GitHub Actions y GitFlow

Colaboración y automatización en el proyecto

4.º del Grado en Ingeniería Informática

Universidad de Las Palmas de Gran Canaria

PAMN · Prácticas · 28 de octubre de 2026

Objetivos de la práctica

- Elegir la rama adecuada para cada cambio.
- Revisar e integrar mediante pull requests.
- Interpretar las comprobaciones de GitHub Actions.
- Preparar una release y propagar un hotfix.
- Justificar la estrategia de ramas del equipo.

Resultado: un repositorio con reglas de colaboración y CI.

Git, GitHub, GitFlow y Actions

Elemento	Función
Git	Registrar cambios y gestionar ramas
GitHub	Alojar el repositorio y revisar propuestas
GitFlow	Acordar cómo organizar las ramas
GitHub Actions	Ejecutar tareas automáticas

La estrategia de ramas y las pruebas se complementan.

CI y CD

Integración continua (CI)

Compilar y probar los cambios con frecuencia.

Entrega continua

Mantener el software preparado para publicar, con posible aprobación manual.

Despliegue continuo

Publicar automáticamente los cambios que superan el proceso.

Las ramas de GitFlow

Rama	Origen	Integración o función
main	Inicialización	Versiones estables
develop	main al comenzar	Próxima release
feature/*	develop	develop
release/*	develop	main y develop
hotfix/*	main	main y develop

Los prefijos expresan un acuerdo. Son ramas normales de Git.

main y develop

main: línea estable

Conserva las versiones preparadas para publicar. Las etiquetas identifican versiones concretas.

develop: línea de integración

Reúne las funcionalidades de la siguiente entrega. Puede avanzar mientras una release se estabiliza.

Una funcionalidad en su propia rama

1. Partir de `develop` actualizado.
2. Crear `feature/contar-tareas`.
3. Añadir el cambio y sus pruebas.
4. Abrir una PR con destino `develop`.
5. Integrar después de CI y revisión.

Una rama por tarea y cambios pequeños facilitan la revisión.

Release y hotfix

	release/*	hotfix/*
Punto de partida	develop	main
Motivo	Preparar una entrega	Corregir la versión estable
Cambios habituales	Ajustes y correcciones	Corrección concreta
Cierre	main y develop	main y develop

La corrección debe llegar también al desarrollo futuro.

Una estrategia según el proyecto

Contexto	Alternativa a valorar
Entregas versionadas y estabilización separada	GitFlow
Equipo pequeño y una versión activa	GitHub Flow
Integración frecuente y cambios muy pequeños	Desarrollo basado en trunk

Actividad: justificar qué coste de coordinación acepta el equipo.

Acuerdo de ramificación del equipo

El archivo `docs/estrategia-ramas.md` debe responder:

- ¿Qué representa cada rama permanente?
- ¿De dónde sale cada cambio y dónde se integra?
- ¿Quién revisa y qué pruebas deben pasar?
- ¿Cómo publicamos y propagamos correcciones?
- ¿Cuándo revisaremos este acuerdo?

Debate en equipo: 10 minutos.

Aplicación del laboratorio

Planificador de tareas en Java 17

- Aplicación de consola con datos en memoria.
- Maven para construir y JUnit para probar.
- Siete pruebas iniciales.
- Una feature añadirá el contador de tareas.

Material: `proyecto-ejemplo.zip`, entregado junto a la guía.

Primera ejecución local

Desde la carpeta que contiene `pom.xml`:

```
java -version
mvn -version
mvn -B -ntp verify
```

Ejecutar la aplicación:

```
java -cp target/classes es.ulpgc.pamn.Planificador "Estudiar Git"
```

Esperado: siete pruebas correctas y una tarea en consola.

Repositorio local y remoto

```
git init -b main
git add .
git commit -m "chore: crear aplicación y CI inicial"
git remote add origin URL-DEL-REPOSITORIO
git push -u origin main
git switch -c develop
git push -u origin develop
```

Crear antes un repositorio **vacío** en GitHub. Sustituir `URL-DEL-REPOSITORIO` por su URL real.

Protección de las ramas permanentes

En `main` y `develop`:

- Exigir pull request y revisión de otra persona.
- Exigir el check `verificar-java`.
- Mantener la rama actualizada antes de integrar.
- Evitar force pushes y el borrado de estas ramas.

El laboratorio utiliza **merge commits**. Una regla de historial lineal impediría estos merges.

Componentes de un workflow

Componente	En el ejemplo
Archivo	.github/workflows/ci.yml
Evento	push o pull_request
Job	verificar
Runner	ubuntu-latest
Steps	Obtener código, preparar Java, ejecutar Maven

Cada job ejecuta sus pasos en un entorno de ejecución.

Workflow: eventos y permisos

```
name: CI Java
on:
  push:
    branches: [main, develop, 'feature/**', 'release/**', 'hotfix/**']
  pull_request:
    branches: [main, develop, 'release/**']
  workflow_dispatch:
permissions:
  contents: read
```

En `pull_request`, el filtro se aplica a la rama **de destino**.

Workflow: job y preparación de Java

```yaml
jobs:
  verificar:
    name: verificar-java
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - name: Obtener el código
        uses: actions/checkout@v7
        with:
          persist-credentials: false
      - name: Preparar Java 17
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: '17'
          cache: maven
```

Workflow: construcción y pruebas

Último step, dentro de la misma lista `steps`:

```
- name: Compilar y ejecutar pruebas
  run: mvn -B -ntp verify
```

- `verify` incluye las fases anteriores, pruebas y empaquetado.
- `-B` evita la interacción y `-ntp` reduce los mensajes.
- Un fallo en las pruebas hace fallar este paso.

El workflow completo está en el proyecto de ejemplo.

Eventos que observaréis en Actions

Acción	Resultado esperado
Push a una feature sin PR	CI del push
Abrir PR hacia develop	CI de la PR
Nuevo push con PR abierta	CI del push y de la PR
Merge en develop	CI del push en develop
Publicar el tag v1.0.0	Ninguna ejecución con este YAML

La duplicación de push y PR es deliberada en el laboratorio.

Feature: contador de tareas

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/contar-tareas
```

Añadir `public int contar()` y dos pruebas:

- Devuelve cero con la lista vacía.
- Devuelve dos después de agregar dos tareas.

Ejecutar `mvn -B -ntp verify` antes del commit.

Pull request de la feature

```
git add src
git commit -m "feat: contar las tareas"
git push -u origin feature/contar-tareas
```

En GitHub, seleccionar:

- **Base:** `develop`.
- **Compare:** `feature/contar-tareas`.

Describir el cambio, indicar las pruebas y pedir revisión.

Una regresión que CI puede detectar

1. Cambiar temporalmente `contar()` para devolver siempre cero.
2. Hacer commit y push a la misma feature.
3. Observar el fallo de la prueba con dos tareas.
4. Corregir el método y enviar otro commit.
5. Comprobar que la PR vuelve a tener el check verde.

Evidencia: enlace a la ejecución fallida y a la corregida.

Revisión e integración

La persona revisora comprueba:

- La rama de destino y el alcance del cambio.
- El comportamiento y los casos de prueba.
- El diff, los archivos generados y las credenciales.

Con aprobación y CI correcta: **Create a merge commit**.

Después: actualizar `develop` y eliminar la feature integrada.

Conflictos durante una integración

Desde una feature que sigue abierta:

```
git fetch origin
git merge origin/develop
```

1. Localizar los archivos con `git status`.
2. Resolver el contenido y retirar los marcadores de conflicto.
3. Hacer `git add`, confirmar el merge y repetir las pruebas.
4. Enviar el resultado con `git push`.

Para cancelar el merge pendiente: `git merge --abort`.

Release 1.0.0: preparación

```
git switch develop
git pull --ff-only origin develop
git switch -c release/1.0.0
```

- Cambiar la versión del proyecto en el POM a `1.0.0`.
- Añadir las notas en `CHANGELOG.md`.
- Ejecutar las pruebas, confirmar y publicar la rama.
- Abrir PR de `release/1.0.0` hacia `main`.

Durante la estabilización, reservar las nuevas features para otra entrega.

Release 1.0.0: etiqueta y propagación

Tras integrar la release en `main` y comprobar su CI:

```
git switch main
git pull --ff-only origin main
git log -1 --oneline
git tag -a v1.0.0 -m "Versión 1.0.0"
git push origin v1.0.0
```

Abrir otra PR de `release/1.0.0` hacia **develop**. Eliminar la release cuando termine esa segunda integración.

Hotfix 1.0.1

Incidencia del ejercicio: rechazar títulos duplicados tras eliminar espacios exteriores, con comparación sensible a mayúsculas.

```
git switch main
git pull --ff-only origin main
git switch -c hotfix/1.0.1
```

Añadir una prueba de regresión, corregir `agregar()` y actualizar la versión y el changelog.

Abrir PR hacia **main** y después hacia **develop**.

Tras el merge y CI en `main`, crear la etiqueta `v1.0.1`. Con una release abierta, integrar también allí el hotfix.

Diagnóstico de una ejecución fallida

En **Actions**, abrir la ejecución y el paso que falla.

Síntoma	Primera comprobación
No aparece ejecución	Ruta del YAML, evento y filtros
Check pendiente	Nombre exigido y cobertura de esa PR
Falla una prueba	Nombre, esperado y obtenido
Solo falla en el runner	JDK, rutas y archivos versionados

Comprobar también la rama y el commit de la ejecución.

Adaptación a la aplicación móvil

Con Gradle Wrapper, las tareas podrían ser:

```
./gradlew testDebugUnitTest lintDebug assembleDebug
```

- Comprobar las tareas de vuestros módulos y variantes.
- Elegir un JDK compatible con Gradle y Android Gradle Plugin.
- Preparar el SDK y distinguir pruebas unitarias e instrumentadas.
- Mantener claves de firma y tokens fuera del historial.

Evidencias para discutir en clase

- Acuerdo de ramas y comparación con otra estrategia.
- PR de feature con revisión de otro integrante.
- Ejecución fallida y corrección comprobada por CI.
- Release etiquetada y cambios propagados a `develop`.
- Hotfix con prueba de regresión e integraciones completas.

Pregunta final: ¿qué simplificaríais para vuestro proyecto y por qué?

Referencias y material de práctica

- Documentación de PAMN facilitada por la coordinadora
- GitHub Actions: conceptos y sintaxis de workflows
- GitFlow: modelo original y reflexión del autor
- GitHub Flow y protección de ramas
- actions/checkout y actions/setup-java

Guía ampliada y aplicación: materiales entregados junto a esta presentación.